

УДК 004.424.74

DOI 10.46418/2079-8199_2021_1_11

Э. А. Мемедляев, К. Н. БусыгинСанкт-Петербургский государственный университет промышленных технологий и дизайна
191186 РФ, Санкт-Петербург, Большая Морская, 18

ИССЛЕДОВАНИЕ ВЛИЯНИЯ ИСПОЛЬЗОВАНИЯ СБОРЩИКОВ МУСОРА НА ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ ВЫПОЛНЕНИЯ JAVA-ПРИЛОЖЕНИЯ

© Э. А. Мемедляев, К. Н. Бусыгин, 2021

В работе получены оценки времени исполнения операций тестового java-приложения в зависимости от используемого сборщика мусора. Исследованы следующие сборщики: Serial GC, Parallel GC, G1 GC, ZGC и Shenandoah GC. Кратко рассмотрены принципы сборки мусора и особенности реализации разных сборщиков. Результаты тестирования получены при запуске приложения с разными объемами доступной памяти и переменным количеством находящихся в памяти «долгоживущих» объектов. Проведен анализ общей производительности приложения, а также перцентилей времени исполнения операций.

Ключевые слова: Java, управление памятью, сборка мусора, SLA.

Постановка задачи

При проектировании Java-приложения предъявляется множество требований, которые должны быть удовлетворены. Одним из таких требований может являться соблюдение соглашения об уровне обслуживания (SLA), задающего, в том числе, ограничение на время отклика программы, то есть на время выполнения операций.

Крайним случаем является создание системы жесткого реального времени, где операция при любых обстоятельствах (в корректно функционирующей системе) не должна занимать больше установленного времени. Такие системы используются, например, на высокоточном производстве, а также в медицинских аппаратах, от функционирования которых зависит жизнь и здоровье человека. Проектирование приложений жесткого реального времени требует использования специализированных инструментов и методов разработки, они должны выполняться в окружении, поддерживающем работу в таком режиме.

Как правило, SLA требует не жесткого, а мягкого реального времени, что, в свою очередь, накладывает более слабые ограничения на время выполнения операций, которые можно в вольной форме выразить как «в подавляющем большинстве случаев операция не должна занимать времени больше установленного, но иногда может».

На возможность выполнения требований мягкого реального времени сильно влияют особенности используемых инструментов, вносящие случайные неоднородные изменения в длительность выполнения операций. Например, операционная система в зависимости от нагрузки процессора может распределять больше процессорного времени другим задачам, уменьшая производительность приложения и увеличивая таким образом время выполнения операций. К тому же эффекту приводит недостаток памяти системы, когда выполнение процесса приводит к активной пагинации,

и значительную часть времени процесс простаивает в ожидании замены страниц памяти.

Также использование системы автоматической сборки мусора вносит свою долю неопределенности из-за непредсказуемости моментов начала и продолжительности циклов сборки мусора. Целью данной работы является исследование влияния процесса сборки мусора на время выполнения операций при использовании в качестве инструментов разработки и исполнения приложений технологии Java. Предполагается, что приложение будет использовать средний объем памяти, не более 8 Гб, и оперировать некрупными объектами.

Краткое описание исследуемых сборщиков мусора

Технологии сборки мусора опираются на гипотезу о поколениях, идея которой в том, что большинство объектов в динамической памяти (куче) используются недолго, и чем дольше объект используется, тем меньше вероятность того, что он перестанет быть нужным впоследствии. Основываясь на этой идее, большинство сборщиков мусора разделяют кучу на два региона: младшее поколение (YoungGen), хранящее порожденные объекты, подавляющее большинство из которых короткоживущие, и старшее поколение (OldGen), предназначенное для «долгожителей». Видов сборки мусора у каждого сборщика тоже по два: малая сборка, включающая только объекты из YoungGen и перемещающая их при необходимости в OldGen, и полная, обрабатывающая еще и (а иногда только) OldGen. Малая сборка выполняется относительно часто по мере заполнения YoungGen и уничтожает большую часть неиспользуемых объектов, тогда как полная сборка обычно запускается тогда, когда в целом свободного места программе становится недостаточно.

Для того чтобы выполнить процедуру сборки, нужно пройти ряд этапов, концептуально схожих во всех реализациях: разметка корневых объектов,

на которые есть ссылки, например, из стеков потоков и статических полей классов; обход графа связей объектов с целью выявления «живых» объектов; выполнение чистки, обычно с помощью перемещения объектов (эвакуации) между регионами во избежание фрагментации; обновление ссылок на объекты.

Некоторые этапы (вроде разметки корней) не могут быть выполнены без остановки всего приложения, что называется остановкой мира (Stop-The-World, STW). Долгие паузы в работе могут крайне негативно сказаться на соблюдении SLA. С другой стороны, часть этапов могут выполняться конкурентно с работой приложения, но за счет общего ухудшения производительности из-за необходимости использования барьеров работы с кучей и перетягивания части ресурсов на себя во время выполнения конкурентных операций. Поэтому различные реализации сборщиков мусора позволяют гибко подойти к настройке временных характеристик управления памятью, что влияет и на все приложение в целом. В работе исследуется влияние пяти сборщиков мусора: Serial GC, Parallel GC, G1 GC, ZGC и Shenandoah GC.

Serial GC существует дольше всех перечисленных, по сравнению с ними он относительно прост. Любая сборка мусора выполняется с остановкой мира в однопоточном режиме. При полной сборке происходит уплотнение OldGen.

Parallel GC по методам работы в память повторяет Serial GC, но старается как можно больше операций выполнять в многопоточном режиме.

G1 GC спроектирован для ограничения времени остановки мира. Оба поколения разбиваются на регионы небольшого размера, и работа по сборке мусора ведется внутри отдельных регионов, что позволяет за один сеанс обработать часть памяти, но до превышения лимита времени остановки мира возобновить выполнение программы. В обычном режиме малой сборки очищаются только блоки YoungGen, но при превышении порога используемой памяти сборка переходит в смешанный режим. При этом сначала в фоновом режиме выполняется цикл пометки, занимающийся выявлением живых объектов. При завершении очередного цикла пометки выполняется смешанная сборка, когда помимо блоков YoungGen очищаются еще и блоки OldGen. Полная сборка выполняется только тогда, когда во время малой сборки нет свободных для эвакуации регионов.

ZGC — экспериментальный сборщик, разработанный для управления огромными кучами, в которых операции по перемещению объектов (при уплотнении, например) занимают неприемлемо большое время. Для минимизации пауз приложения остановки мира требуют только этапы работы с корнями. Все остальные происходят в конкурентном режиме, в том числе и перемещение объектов в куче. Из-за того, что во время работы программы объекты могут внезапно поменять адрес, используется раскраска ссылок для отображения текущего состояния объекта в битах его адреса и барьер загрузки, срабатывающий каждый раз при использовании ссылки на кучу, проверяющий состояние объекта

и выполняющий при необходимости перемещение объекта прямо на месте. В итоге паузы остановки мира минимизируются, но конкурентное перемещение объектов и использование барьера загрузки равномерно снижает производительность программы.

Shenandoah GC — также экспериментальный сборщик, концептуально похожий и на G1, и на ZGC. Он не использует разделение кучи на поколения, но разбивает ее на некрупные регионы, работу по разметке и эвакуации объектов проводит в конкурентном режиме с недолгими паузами на инициализацию и завершение, для обеспечения непротиворечивости работы с кучей использует блокировки чтения, записи и эквивалентности.

Параметры тестового стенда

Для целей исследования разработано тестовое приложение, которое в качестве базовой операции выполняет преобразование HTML-страниц в объектную модель. Преобразование выполняется пакетом jsoup-1.13.1. HTML-страницы взяты из openjdk 15.0.1 javadoc в количестве 10133 файлов, за время одного испытания все файлы подвергаются преобразованию 50 раз, что приводит к выполнению операции 506650 раз. Для параллельного выполнения преобразований порождается 12 потоков исполнения (рабочих потоков). При выполнении каждой операции очередной файл считывается с жесткого диска, выполняется преобразование, результат не сохраняется.

Из-за того, что результат преобразования не сохраняется, каждая операция приводит к порождению большого количества короткоживущих объектов, которые будут уничтожены при первой же сборке мусора. Для моделирования использования приложением некоторого количества долгоживущих объектов каждый рабочий поток располагает кольцевым буфером, заполненным фиктивными объектами определенного размера (около 2 Кб). Изменяя размер кольцевого буфера, можно регулировать количество долгоживущих объектов. В конце исполнения каждой операции часть объектов в кольцевом буфере заменяется на новые, обеспечивая таким образом ограниченное время жизни долгоживущих объектов и постоянное перемещение объектов между поколениями при работе сборщика мусора. Но при этом стоит отметить, что количество объектов, подлежащих уплотнению, в целом постоянно.

Для сравнения использовались пять сборщиков мусора — Serial GC, Parallel GC, G1 GC, ZGC и Shenandoah GC. Тестовые программы запускались на OpenJDK 64-Bit Server VM (build 14.0.2+12-Ubuntu-120.04, mixed mode, sharing). Тонкая настройка с помощью задания параметров сборщиков не проводилась, использовались значения по умолчанию.

Для получения дополнительной информации исполняемые тесты профилировались с использованием Flight Recorder и perf, что приводило к незначительным равномерным потерям в производительности. Дополнительно собирались данные по использованию сборщика мусора штатными средствами логирования JVM, для анализа которых использовался сервис GCeasy.

Выполнение тестов производилось на операционной системе Ubuntu 20.04.1 LTS. Запуск тестов производился без активных сторонних процессов. Чтобы при больших объемах требуемой для процесса памяти отследить момент бробуксовки и активную пагинацию, которая может внести сильный вклад во время выполнения операций, с помощью perf контролировалось общее количество ошибок страницы за время выполнения программы.

Тесты проводились на компьютере Intel (R) Core (TM) i3-8130U CPU @ 2.20GHz, 8Гб DDR4.

Результаты измерений и их обсуждение

Было проведено два набора тестов. Итоговые результаты анализировались с целью сравнения значений 95-, 99-, 99.9- и 99.99-перцентилей времен выполнения операций. Результаты были получены для сильно упрощенной модели программы, и в реальных проектах зависимости могут отличаться, но это не мешает наметить общие тенденции. Также стоит отметить, что длительность одной из самых затратных операций — перемещения объектов в куче — не зависела от количества долгоживущих объектов, а определялась константой.

Первый набор тестов заключался в запуске программ, не имеющих большого количества долгоживущих объектов и активно генерирующих короткоживущие объекты. Варьировался объем доступной процессу памяти и используемый сборщик мусора. Производительность тестового приложения приведена на рисунке 1, перцентили времени выполнения операций — на рисунке 2.

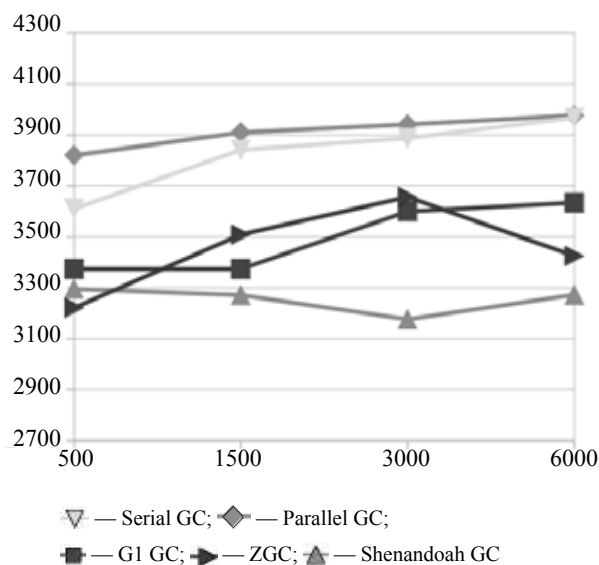


Рис. 1. Зависимости производительности приложения (базовых операций/сек) от объема доступной процессу памяти (Мб) при выполнении тестовой программы с разными сборщиками мусора

В случае, когда программа имеет дело с короткоживущими объектами, то ни объем памяти, ни используемый сборщик мусора не вносят существенного вклада во время выполнения операций для всех сборщиков мусора. Генерируемые объекты остаются в YoungGen, не переходя в старшие поколения и не вызывая сложную полную сборку мусора.

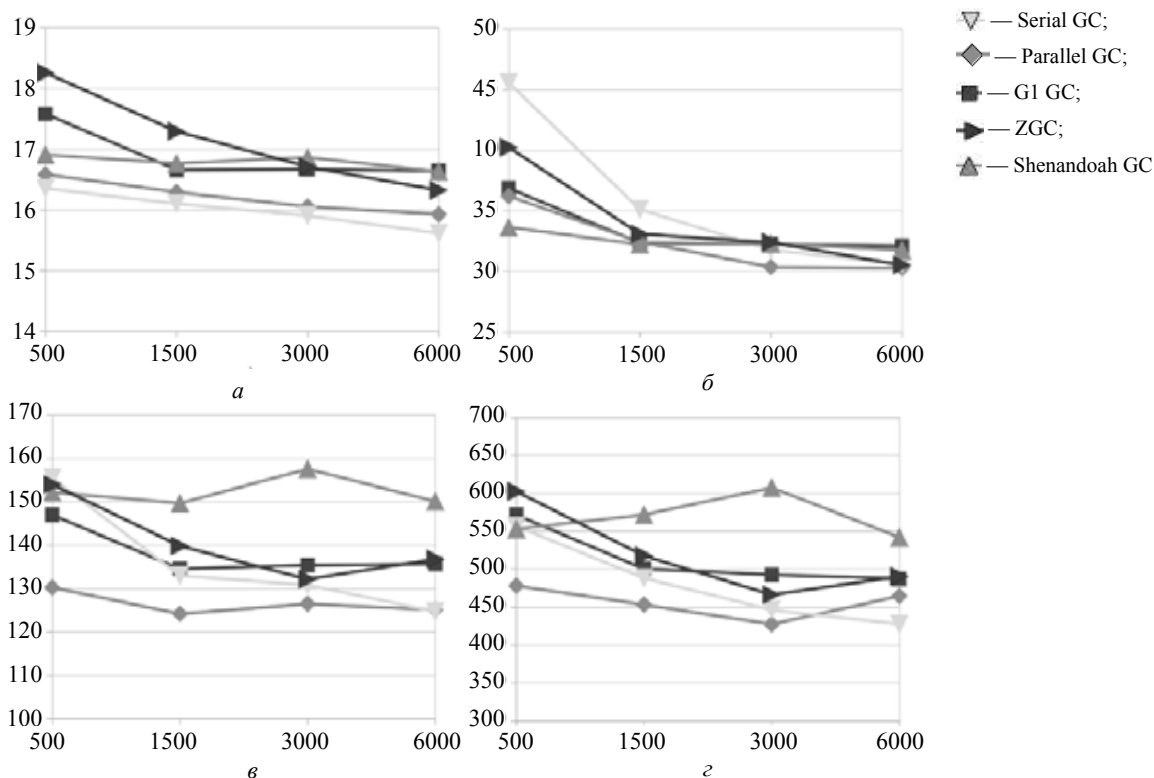


Рис. 2. Зависимости перцентилей времени выполнения операций (мс) от объема доступной процессу памяти (Мб) при выполнении тестовой программы с разными сборщиками мусора. а) — 95-перцентиль; б) — 99-перцентиль; в) 99.9-перцентиль; г) 99.99-перцентиль

Производительность программы при увеличении доступной памяти слегка увеличивается, что заметно для 95- и 99-перцентилей, из-за увеличения YoungGen и, соответственно, уменьшения общего количества сборок мусора, имеющих значительные временные расходы на инициализацию и завершение.

Сборщики мусора, не имеющие конкурентных фаз работы, выигрывают в производительности за счет отсутствия связанных с таким режимом накладных расходов и выполнения в процессе работы только быстрых легких сборок.

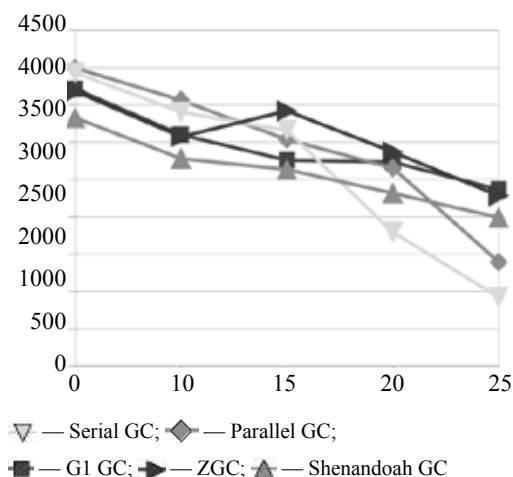


Рис. 3. Зависимости производительности приложения (базовых операций/сек) от количества долгоживущих объектов в куче (10^5 ед.) при выполнении тестовой программы с разными сборщиками мусора

Программы из второго набора тестов так же активно порождали короткоживущие объекты, но дополнительно имели живые ссылки на большое фиксированное количество долгоживущих объектов. Варьировались количество долгоживущих объектов и используемый сборщик мусора. Производительность тестового приложения при запусках второго набора приведена на рисунке 3, перцентили времени выполнения операций — на рисунке 4.

При отсутствии серьезной работы по управлению памятью лидерство держат сборщики, работающие в режиме остановки мира. Увеличение размера кучи ожидаемо ухудшает производительность приложения из-за увеличения накладных расходов на разметку объектов. Причем в этой ситуации ощутимо выигрывают сборщики, выполняющие разметку в конкурентном режиме.

Увеличение количества управляемых объектов приводят к ухудшению временных показателей времени выполнения. И если для 95-перциля разница невелика, то на остальных видно, что выбор сборщика мусора может существенно снизить отклик программы.

Serial GC быстро перестает справляться с растущей кучей. Если для 95- и 99-перцентилей время выполнения находится на уровне, сопоставимом с остальными, то дальше значения резко идут вверх, что означает невозможность применения при жестких SLA.

Parallel GC очень хорошо себя проявляет для большинства времен исполнения операций за счет параллельной сборки мусора, но крайние значения, как и у Serial GC, большие — нет контроля над време-

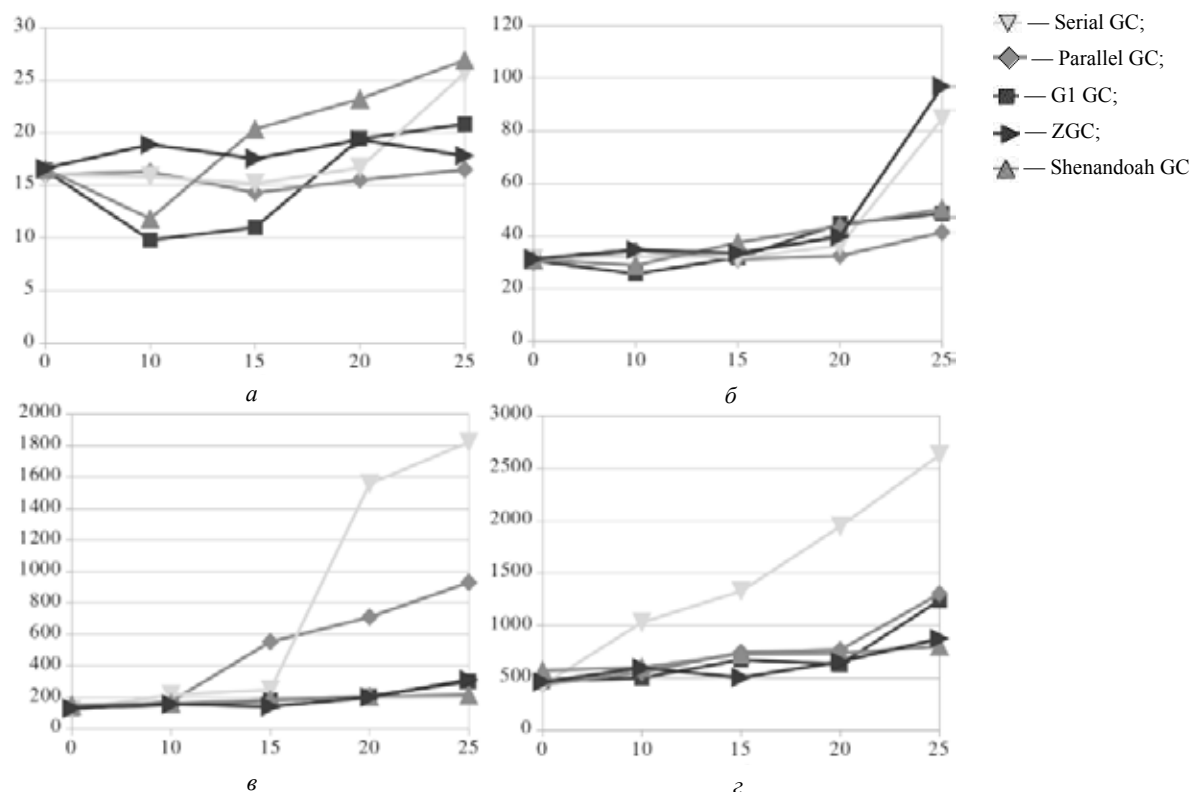


Рис. 4. Зависимости перцентилей времен выполнения операций (мс) от количества долгоживущих объектов в куче (10^5 ед.) при выполнении тестовой программы с разными сборщиками мусора. а) — 95-перцентиль; б) — 99-перцентиль; в) 99.9-перцентиль; г) максимальные времена выполнения

нем одного сеанса сборки, пауза остановки мира при полных сборках велика.

G1 GC умело балансирует и показывает хороший результат на всех перцентилях, лидируя либо держась ближе к лучшей в категории группе. Исключение составляет 99.99-перцентиль при максимальной загрузженности долгоживущими объектами, где он начинает выбиваться из лучшей группы из-за нехватки свободной памяти для комфортной работы, что приводит к регулярным сеансам долгой полной сборки мусора, выполняющейся аналогично алгоритмам Parallel GC.

ZGC и Shenandoah ведут себя очень похоже, поскольку предназначены для управления большими кучами, поэтому их алгоритмы не добиваются лучшего времени работы для большинства операций, но прекрасно себя проявляют в крайних значениях — наибольшее время исполнения сильно ниже, чем у остальных.

Среднее время остановки мира при работе разных сборщиков мусора приведено в таблице 1. Интересно отметить, что те сборщики мусора, которые созданы для сокращения времени остановки мира при работе с большими кучами, очень хорошо справляются со своей задачей (особенно ZGC, таблица 1), что, впрочем, не всегда приводит к общему уменьшению времени выполнения операций.

Выводы

Выбор сборщика мусора может помочь провести тонкую настройку временных характеристик управления памятью, что влияет и на время выполнения операций приложения в целом.

Распределение времени исполнения операций Java-приложения, не хранящего в памяти большое количество долгоживущих объектов, но активно порождающего короткоживущие, не сильно зависит от того, какой сборщик мусора используется. С другой стороны, производительность приложения при использовании более интеллектуальных сборщиков будет ниже из-за увеличенных накладных расходов. Также вне зависимости от используемого сборщика увеличение объема доступной памяти влечет некоторое увеличение производительности программы за счет сокращения общего количества сборок мусора.

Таблица 1. Среднее время остановки мира сборщиков мусора при работе с кучей, содержащей разное количество долгоживущих объектов, мс.

	Количество долгоживущих объектов, 10 ⁵				
	0	10	15	20	25
Serial GC	19,20	281,00	437,00	802,00	1463,00
Parallel GC	7,97	145,00	529,00	694,00	876,00
G1 GC	15,30	62,50	53,00	72,50	129,00
Z GC	0,21	0,16	0,15	0,15	0,15
Shenandoah GC	0,45	0,37	0,33	0,33	40,40

Временные характеристики работы приложения, управляющего не самой большой кучей и имеющего в наличии достаточное количество свободной памяти, как и в первом случае, не сильно зависят от используемого сборщика мусора. Но при увеличении памяти даже при относительно небольших размерах кучи (около 5,2 Гб) преимущество получают более интеллектуальные механизмы управления памятью. Для обеспечения требуемых характеристик программы стоит отказаться от сборщика мусора по умолчанию и провести тесты с альтернативными. В данной работе исследовались возможности сборщиков с установленными по умолчанию настройками, но подстройка параметров под особенности конкретного приложения может дать существенное улучшение требуемых характеристик выполнения.

Список литературы. References

1. Jones R. The garbage collection handbook: the art of automatic memory management / Jones R., Hosking A., Moss E. Boca Raton: CRC Press, 2012. 514 p.
2. Memory Management in the Java HotSpot™ Virtual Machine. URL: <https://www.oracle.com/technetwork/java/javase/memorymanagement-whitepaper-150215.pdf> (дата обращения: 25.01.2021).
3. Getting Started with the G1 Garbage Collector. URL: <https://www.oracle.com/technetwork/tutorials/tutorials-1876574.html> (дата обращения: 25.01.2021).
4. An Introduction to ZGC. URL: <https://www.baeldung.com/jvm-zgc-garbage-collector> (дата обращения: 25.01.2021).
5. Experimental Garbage Collectors in the JVM. URL: <https://www.baeldung.com/jvm-experimental-garbage-collectors> (дата обращения: 25.01.2021).
6. Shenandoah GC. URL: <https://wiki.openjdk.java.net/display/shenandoah> (дата обращения: 22.12.2021).

Е. А. Мемедляев, К. Н. Бусыгин

Saint Petersburg State University of Industrial Technologies and Design
191186 Russian Federation, Saint Petersburg, Bolshaya Morskaya str., 18

INVESTIGATION OF THE IMPACT OF USING GARBAGE COLLECTORS ON THE RUNTIME CHARACTERISTICS OF A JAVA APPLICATION

In this paper, we obtain estimates of the execution time of operations of a test java application, depending on the garbage collector used. The following assemblers were investigated: SerialGC, Parallel GC, G1 GC, GC, and Shenandoah GC. The principles of garbage collection and the implementation features of different collectors are briefly considered. The test results are obtained when running an application with different amounts of available memory and a variable number of «long-lived» objects in memory. The analysis of the overall performance of the application, as well as the percentiles of the execution time of operations, is carried out.

Keywords: Java, memory management, garbage collection, SLA.